# Web and Data Engineering

## Vorlesung 07: Sonstiges — Vertiefungen

Prof. Dr. Michael Granitzer

# Sonstiges

Diese Einheit sammelt Vertiefungen, die in den bisherigen Vorlesungen zu detailliert gewesen wären, aber im Verständnis moderner Web-Systeme hilfreich sind.

## Fokus

- wie JavaScript intern arbeitet (Event Loop, asynchrone Ausführung)
- warum scheinbar einfache Seiten langsam sein können (Rendering- und Ladestrategien)

# JavaScript-Laufzeit

**Leitfrage:** Wie führt der Browser JavaScript aus — und warum ist das Ausführungsmodell (single-threaded, ereignisbasiert) die Ursache fast aller Performance- und Reihenfolge-Überraschungen?

# Was gibt dieser Code aus?

```
console.log("1: Start");

setTimeout(() => console.log("2: Timeout"), 0);

fetch('/api/products/42')
  .then(response => response.json())
  .then(data => console.log("3: Daten geladen"));

console.log("4: Ende");
```

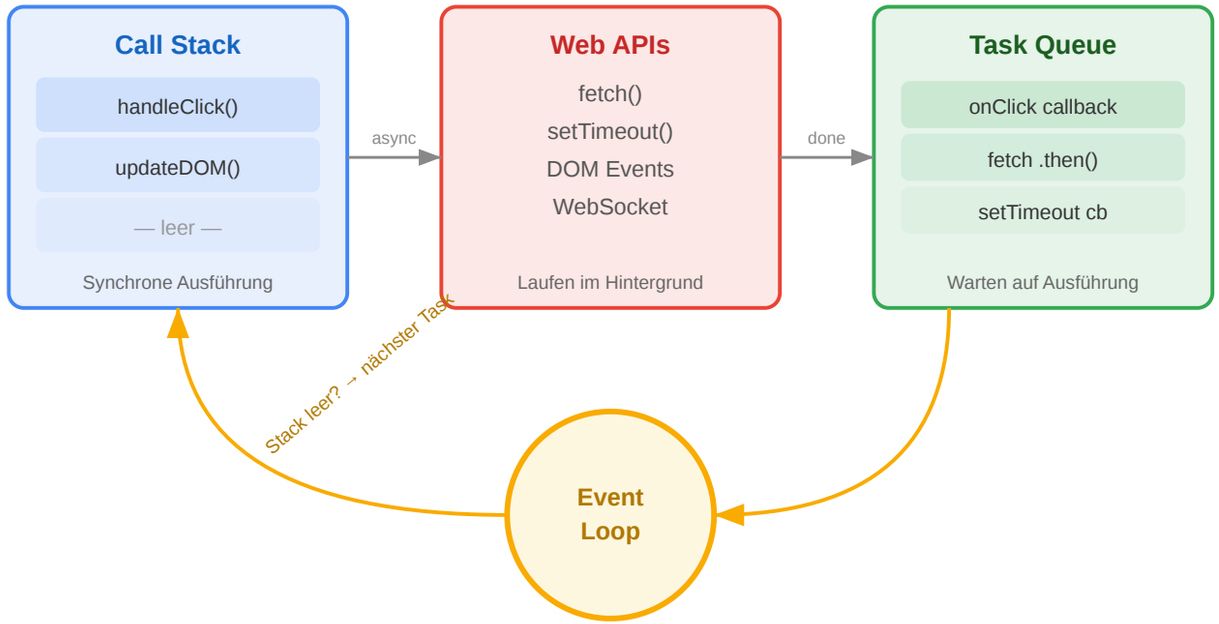
**Aufgabe:** In welcher Reihenfolge erscheinen die vier Zeilen in der Konsole? Begründen Sie.

# Auflösung

`1: Start → 4: Ende → 2: Timeout → 3: Daten geladen`

JavaScript ist **single-threaded —** aber `setTimeout` und `fetch` laufen im Hintergrund. Der aktuelle Code wird zuerst vollständig ausgeführt, dann kommen die Callbacks. Das ist das fundamentale Ausführungsmodell, das Web-Anwendungen performant macht.

# Der Event Loop: Warum das Web nicht blockiert




## Wie es funktioniert

1. **Call Stack:** Führt den aktuellen Code synchron aus
2. **Web APIs:** Browser übernimmt Netzwerk, Timer, Events im Hintergrund
3. **Task Queue:** Fertige Callbacks warten hier
4. **Event Loop:** Wenn der Call Stack leer ist, holt er den nächsten Callback

## Warum das architektonisch wichtig ist

- Ein **einziger Thread** für UI und Logik → blockierende Operationen frieren die Seite ein
- Deshalb ist **alles I/O asynchron** (Netzwerk, Timer, File API)
- Node.js nutzt dasselbe Modell serverseitig → ein Thread bedient tausende Verbindungen

# Warum ist diese Seite langsam?

**Leitfrage:** Warum lädt eine scheinbar einfache Seite mehrere Sekunden — und welche Reihenfolge von Ressourcen im <head> entscheidet über den Rendering-Start?

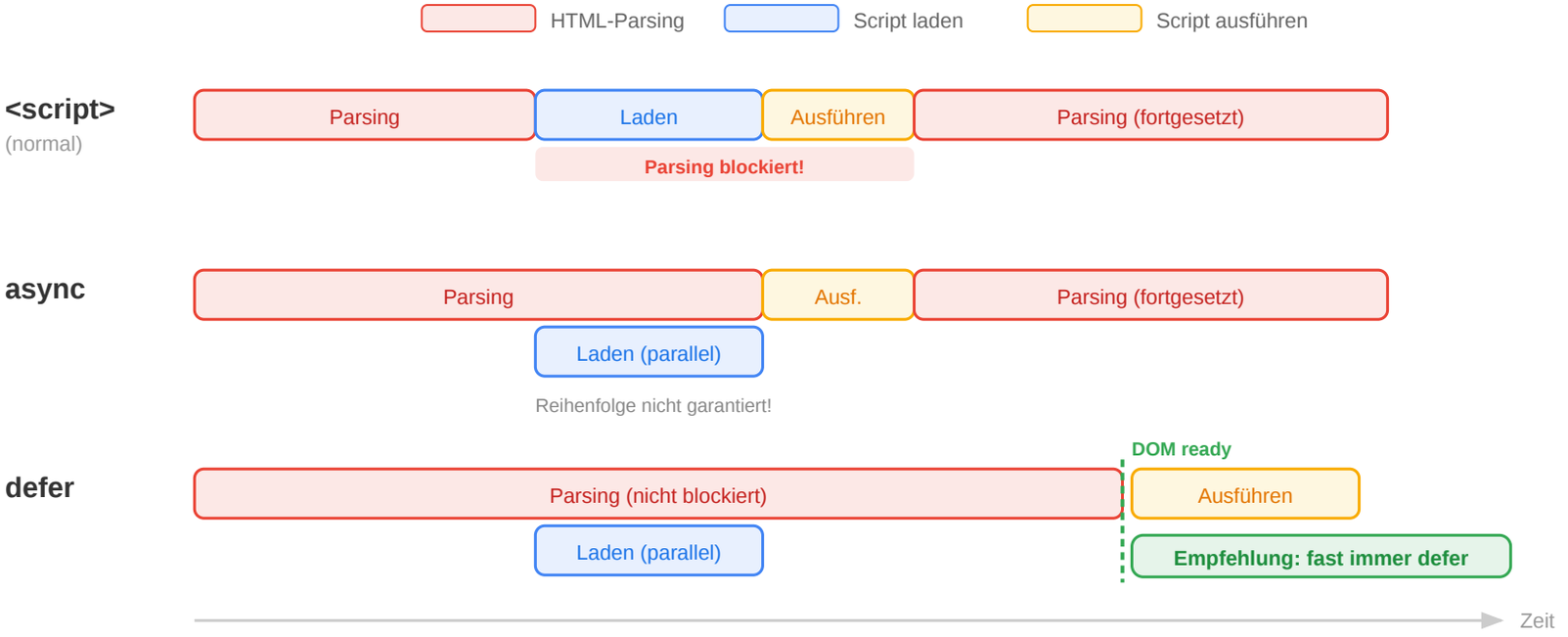
# Warum ist diese Seite langsam?

**Aufgabe:** Eine Produktseite des Online-Shops lädt in 4,2 Sekunden. Der Entwickler hat den HTML-Head so strukturiert:

```
<head>
  <script src="analytics.js"></script>
  <script src="app.js"></script>
  <link rel="stylesheet" href="styles.css">
  <script src="chat-widget.js"></script>
</head>
```

Was ist das Problem — und wie würden Sie es lösen?

# Script-Ladestrategien



# Analyse

| Problem | Ursache | Lösung |
|---|---|---|
| **Scripts blockieren Parsing** | `<script>` ohne `defer`/`async` stoppt den HTML-Parser | `<script src="app.js" defer>` |
| **CSS kommt nach Scripts** | Browser wartet auf Scripts, bevor er CSS sieht | CSS **vor** Scripts im `<head>` |
| **Alles synchron geladen** | 3 Scripts + 1 CSS = sequenzielle Blockade | `defer` für alle Scripts, CSS zuerst |

## Optimierte Version

```
<head>
  <link rel="stylesheet" href="styles.css">
  <script src="app.js" defer></script>
  <script src="analytics.js" defer></script>
  <script src="chat-widget.js" defer></script>
</head>
```

`defer` = Script wird **nach** dem Parsen ausgeführt, **in Reihenfolge**. Das ist fast immer die richtige Wahl.

# Wrap-Up

- Der **Event Loop** erklärt, warum JavaScript-Code nicht in der Reihenfolge läuft, in der er geschrieben ist: synchroner Code zuerst, Callbacks danach, Microtasks vor Tasks.
- **Ladezeit-Probleme** im Frontend entstehen oft nicht *im* Code, sondern an der Reihenfolge von Ressourcen: render-blocking Scripts, CSS nach Scripts, fehlendes `defer`.
- Beides sind Vertiefungen zum Zusammenspiel der Technologien, das in Vorlesung 03 umrissen wurde.